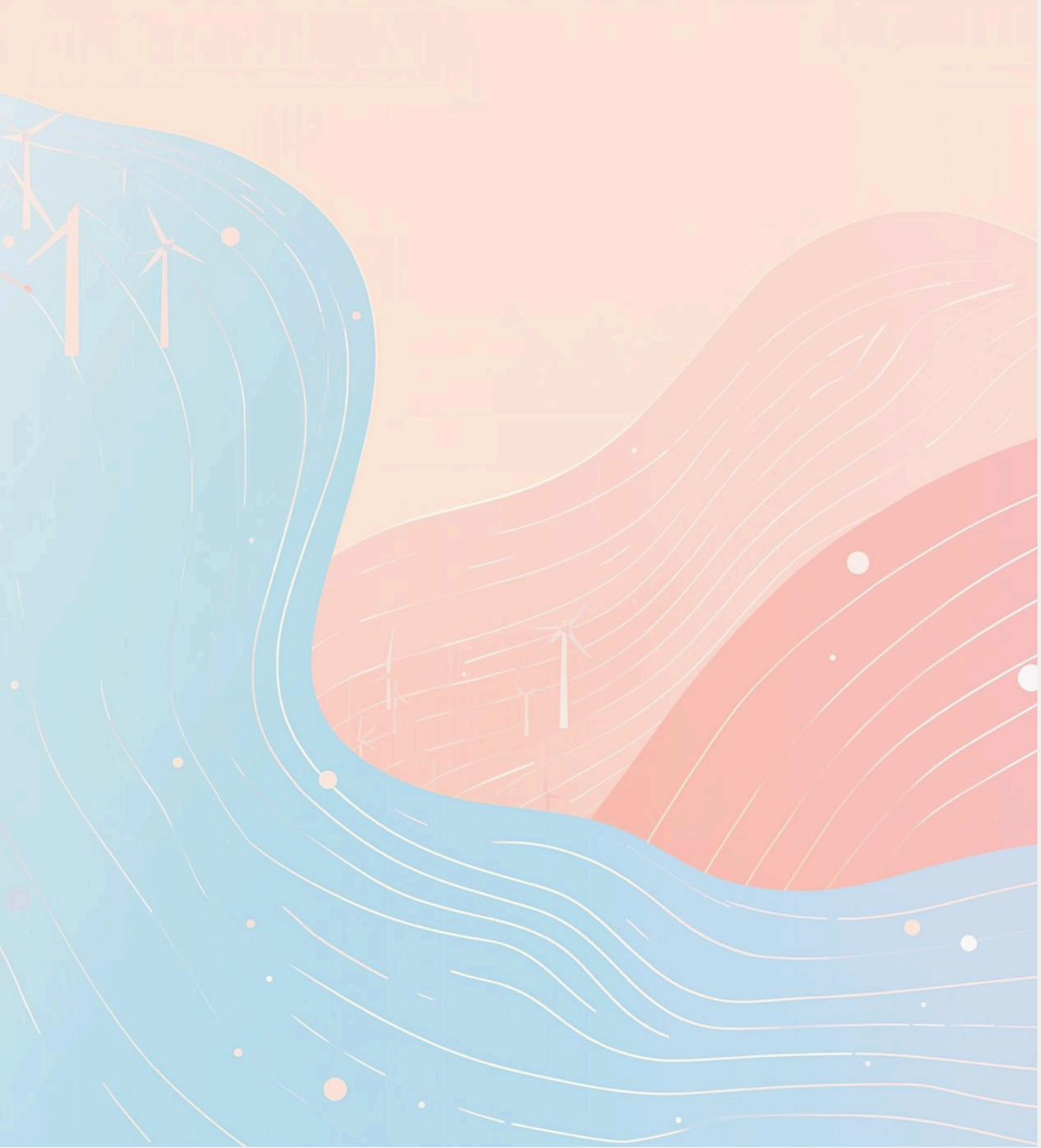


Energy Insights

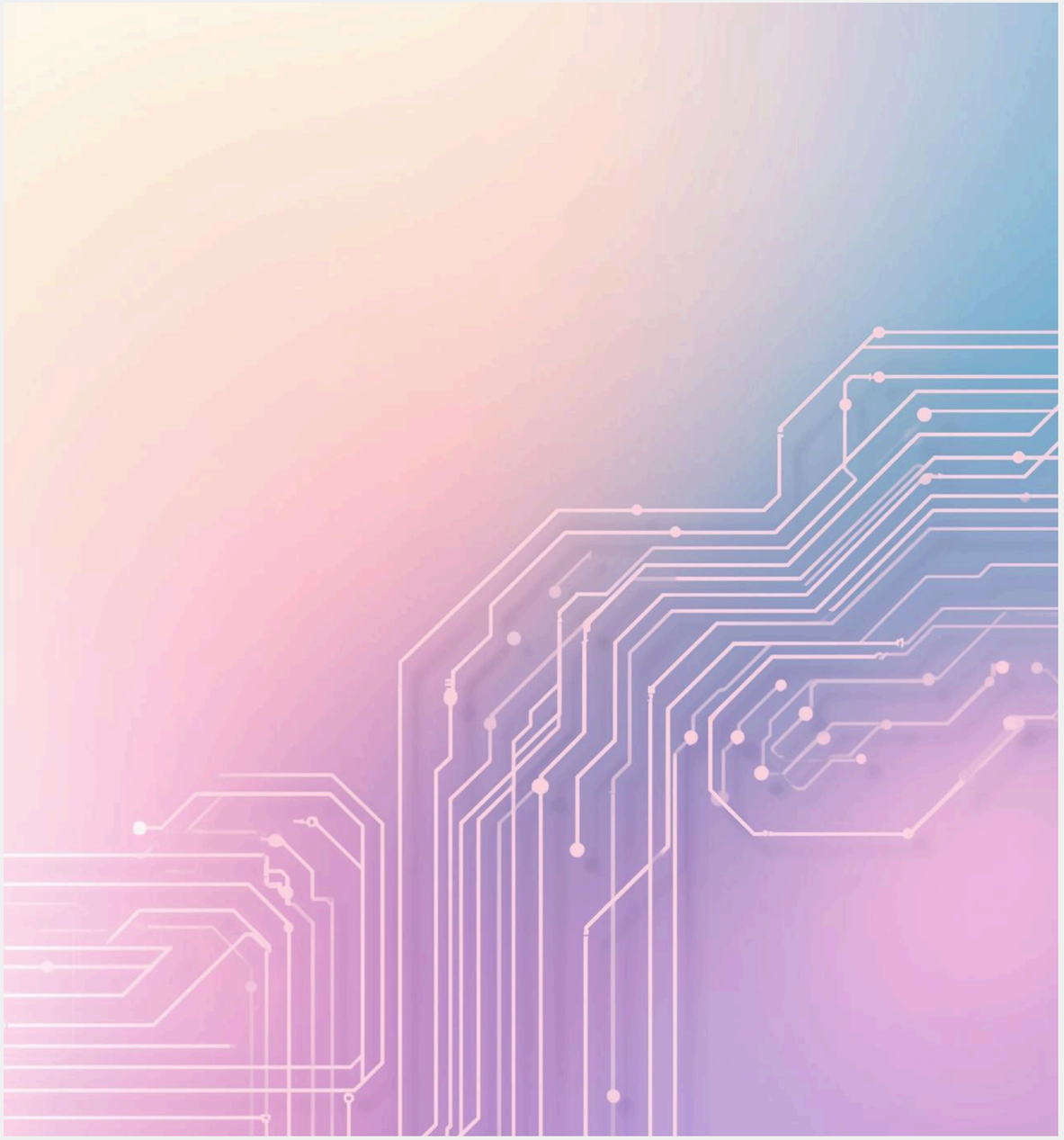
Predicting Building Energy Consumption

DATA SCIENCE - GROUP 8

Project Overview

UNDERSTANDING ENERGY CONSUMPTION PATTERNS

Our project builds a machine learning pipeline to predict building energy consumption considering weather and building data. The workflow includes Exploratory Analysis (EDA), data preprocessing, feature engineering, model training, and performance evaluation to identify the most accurate predicting approach.



Exploratory Data Analysis

Dataset Overview

UNDERSTADING OUR DATA

Source: Building energy consumption dataset (Kaggle)

Size:

- 20, 000 observations
- 18 variables of which 16 features and one identifier

Target variable

- **energy_consumption** → continuous value representing building energy consumption.

Feauture variables: building characteristics, environmental & weather, contextual/surroundings, operational.

Types of data: numerical, categorical (nominal and ordinal), metadata.

```
<class 'pandas.core.frame.DataFrame
RangeIndex: 20000 entries, 0 to 199
Data columns (total 18 columns):
 #   Column              Non-Null C
---  ------              -----------
 0   id                  20000 non-
 1   temp_outdoor        17434 non-
 2   humidity            16372 non-
 3   insulation          20000 non-
 4   heating_type        20000 non-
 5   occupancy           20000 non-
 6   floor_area          18686 non-
 7   wall_thickness      20000 non-
 8   roof_age            20000 non-
 9   window_age          20000 non-
 10  basement_depth      20000 non-
 11  garden_size         20000 non-
 12  solar_irradiance    20000 non-
 13  wind_exposure       20000 non-
 14  air_quality_index   20000 non-
 15  street_noise_db     20000 non-
 16  construction_year   20000 non-
 17  energy_consumption  20000 non-
dtypes: float64(13), int64(3), obje
memory usage: 2.7+ MB
```

Data Quality Issues

UNDERSTADING OUR DATA

MISSING VALUES

- humidity ≈ 18% missing
- temp_outdoor ≈ 13% missing
- floor_area ≈ 7% missing

Implication → Imputation is required before modeling.

MIXED DATA TYPES

- Numerical: temperature, humidity, floor_area, etc.
- Categorical: insulation, heating_type
- Identifier: id

Implications →Encode categorical variables; Drop id to avoid data leakage

HETEROGENOUS FEATURE SCALES

- floor_area ranges ~40 to 300
- occupancy ranges 1 to 5
- Weather variables in small numeric ranges

Implication → Feature scaling is recommended for scale-sensitive models.

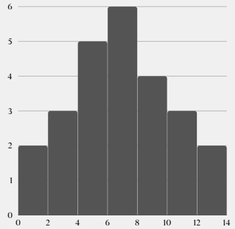
INVALID TARGET VALUES

- energy_consumption contains negative values (min ≈ -9.6)

Implication → negative energy consumption to be either cleaned or transformed

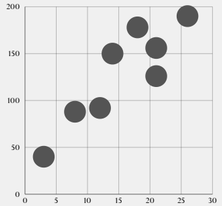
EDA Key Insights I

UNDERSTADING OUR DATA



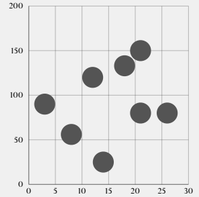
TARGET VARIABLE DISTRIBUTION

Distribution is approximately bell-shaped, showing a roughly normal distribution. It has a mean ≈ 20 kWh, with moderate spread. Some extreme low and high values exist.



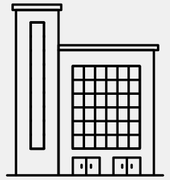
STRONGEST CORRELATIONS

include **humidity**, temp_outdoor, occupancy and floor_area. These features are expected to be meaningful predictors.



WEAKEST CORRELATIONS

include roof_age, basement_depth, street_noise_db, garden_size. These features may remain for tree-based models. Feature importance will determine their actual usefulness.



BUILDING SIZE MATTERS

Larger floor_area generally corresponds to higher energy usage; which confirms physical intuition and dataset feasibility..

EDA Key Insights II

UNDERSTADING OUR DATA

OCCUPANCY AS AN ORDINAL FEATURE

Discrete values from 1 to 5, making it better suited for non-linear models.

INSULATION IMPACTS CONSUMPTION

Mean energy consumption decreases from low → medium → high insulation

HEATING TYPES SHOW MODERATE DIFFERENCES

Electric heating has a slightly higher average than gas and heat pump.

CATEGORICAL VARIABLES ARE REASONABLY BALANCED

- Insulation: ~50% medium, ~30% low, ~20% high
- Heating type: ~40% gas, ~40% electric, ~20% heat pump

Reducing risk of biased learning after encoding.

NMAR Analysis

NOT MISSING AT RANDOM

MISSING VALUES

- humidity ≈ 18% missing
- temp_outdoor ≈ 13% missing
- floor_area ≈ 7% missing

Implication → Imputation is required before modeling.

WHAT IS NMAR?

Missingness depends on the unobserved value itself (e.g., sensors failing under extreme conditions).

WHY IT MATTERS?

Missing values may carry information about energy usage and should be explicitly modeled.

ANALYSIS APPROACH

Compare average energy consumption when a feature is missing versus present; large differences suggest creating a missingness indicator.

Data Preprocessing & Hyperparameter tuning

TUNNING FEATURES AND MODEL PARAMETERS

NMAR

HANDLING MISSING DATA

```
================================================================================
NMAR ANALYSIS: Impact of Missing Data on Energy Consumption
================================================================================
         col  missing_rate  mean_y_missing  mean_y_present  diff_missing_minus_present
temp_outdoor        0.1283 =12,8% 25.968676       19.292758                    6.675917
    humidity        0.1814 = 18,1% 21.449480       19.861157                    1.588323
  floor_area        0.0657 =6,6% 20.486952       20.125533                    0.361419

 Interpretation:
Buildings with missing values consume MORE energy
```

+6.68 kWh
+1.59 kWh
+0.36 kWh

Step 1

- Detect missing values

Step 2

- Create binary indicators (...._is_missing)

Step 3

- Median imputation

Feature engineering

PHYSICS-BASED FEATURES

Weather Features

- temp_sq: Quadratic temperature (captures non-linear heating/cooling)
- HDD: Heating Degree Days = max(18°C - temp, 0)
- CDD: Cooling Degree Days = max(temp - 18°C, 0)
- wind_x_temp: Wind amplifies temperature effects

Building Age

- building_age = 2026 - construction_year
- construction_year

(kept both!)

```
Manual Weights Applied: CatBoost=0.4, XGBoost=0.3, Ridge=0.2, RF=0.1
```

With only building_age: RMSE increased to 3.33.
With only construction_year: RMSE was 3.31.
With both: RMSE stayed at 3.31

Interactions

- temp_x_insulation: Insulation matters more in extreme temps
- temp_x_wall: Wall thickness provides temperature buffer
- solar_x_window_age: Old windows lose solar heat gain
- wall_per_area: Thermal mass per unit area
- occupancy_density: People per square meter

Density Features

- occupancy_density: People per square meter
- wall_per_area: Structural density or thermal mass relative to the building's size

Categorical encoding

CATEGORICAL FEATURES

Insulation (Ordinal)

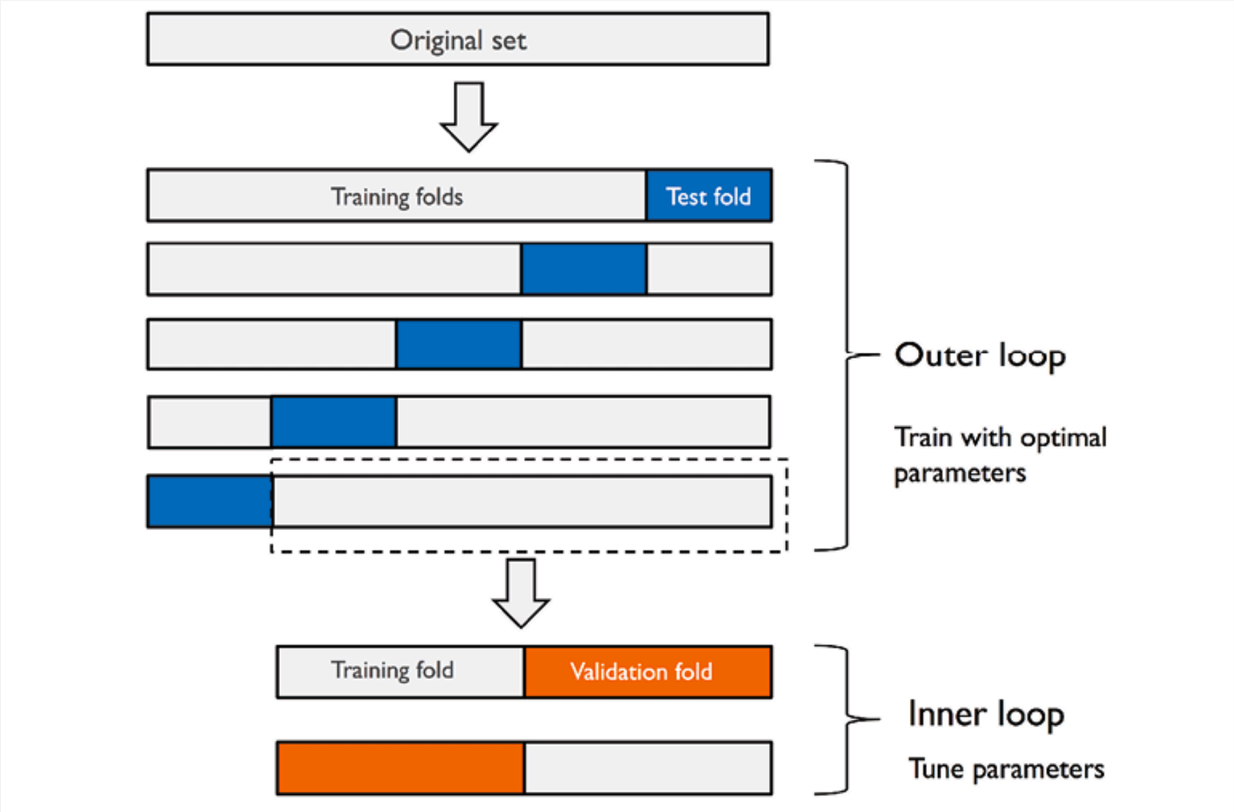
low → 0
medium → 1
high → 2

Heating Type (OHE)

Before	After		
heating_type	heat_gas	heat_electric	heat_heatpump
gas	1	0	0
electric	0	1	0
heatpump	0	0	1

Nested CV

HYPERPARAMETER TUNING: NESTED CV STRATEGY



Grid Search
'best of the best'

hyperparameter search

Grid search Setup

CatBoost (32 combinations)

depth: [4, 6]
learning_rate: [0.03, 0.06]
l2_leaf_reg: [3, 7]
iterations: [600, 900]

XGBoost (64 combinations)

max_depth: [4, 6]
learning_rate: [0.03, 0.05]
subsample: [0.8, 1.0]
colsample_bytree: [0.8, 1.0]
n_estimators: [400, 800]

Ridge

alpha: [0.5, 2.0, 10.0]

Random Forest

n_estimators: [200, 500]
max_depth: [8, 12]
min_samples_leaf: [1, 3]

Model Choice, Ensembling & Final Model Decision

FROM EXPLORATION TO ROBUST GENERALIZATION

Models Considered & Initial Strategy

We evaluated multiple supervised learning models, focusing mainly on tree-based and ensemble methods due to the presence of nonlinear relationships, feature interactions, and missing values.

Models evaluated:

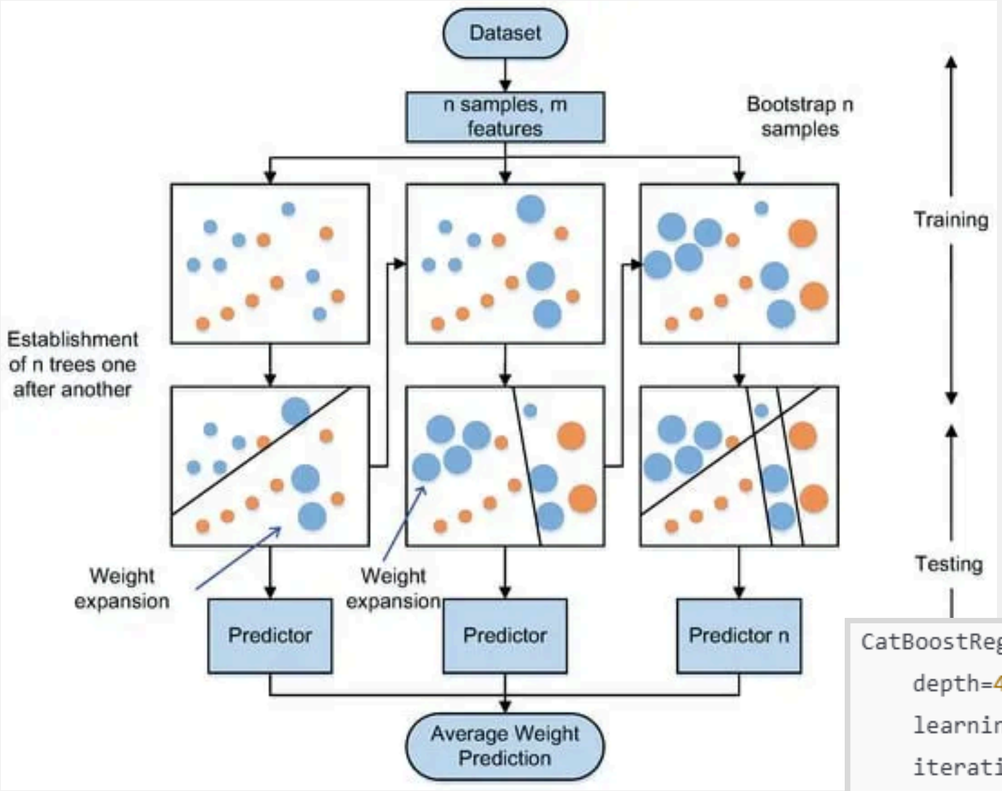
- CatBoost: Strong with categorical features, handles missing data
- XGBoost: Powerful gradient boosting, good regularization
- Random Forest: Ensemble of trees, robust to outliers
- Ridge Regression: Linear baseline, interpretable

Goal: balance predictive power, robustness, and generalization

```python
# --- Model 1: CatBoost ---
model_cb = CatBoostRegressor(
    **best_params_cb,
    verbose=0,
    random_seed=RANDOM_STATE,
    loss_function="RMSE"
)
model_cb.fit(X_tr_numeric, y_tr, eval_set=(X_va_numeric, y_va), early_stopping_rounds=50)
oof_cb[va_idx] = model_cb.predict(X_va_numeric)
test_cb += model_cb.predict(X_test_numeric) / N_SPLITS
models_cb.append(model_cb)

# --- Model 2: Ridge ---
model_ridge = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge(alpha=best_params_ridge.get("ridge__alpha", 2.0)))
])
model_ridge.fit(X_tr_numeric, y_tr)
oof_ridge[va_idx] = model_ridge.predict(X_va_numeric)
test_ridge += model_ridge.predict(X_test_numeric) / N_SPLITS
models_ridge.append(model_ridge)

# --- Model 3: Random Forest ---
model_rf = RandomForestRegressor(
    **best_params_rf,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
model_rf.fit(X_tr_numeric, y_tr)
oof_rf[va_idx] = model_rf.predict(X_va_numeric)
test_rf += model_rf.predict(X_test_numeric) / N_SPLITS
models_rf.append(model_rf)

# --- Model 4: XGBoost ---
model_xgb = XGBRegressor(
    **best_params_xgb,
    random_state=RANDOM_STATE,
    early_stopping_rounds=50,
    eval_metric="rmse"
)
model_xgb.fit(X_tr_numeric, y_tr, eval_set=[(X_va_numeric, y_va)], verbose=False)
oof_xgb[va_idx] = model_xgb.predict(X_va_numeric)
test_xgb += model_xgb.predict(X_test_numeric) / N_SPLITS
models_xgb.append(model_xgb)
```

CatBoost

HOW CATBOOST WORKS AND WHY IT FITS THE DATA




```
CatBoostRegressor(
    depth=4,
    learning_rate=0.03,
    iterations=600,
    loss_function="RMSE"
)
```

Gradient Boosting with Decision Trees

- Builds an ensemble of shallow decision trees
- Each tree corrects errors made by previous trees

Ordered Boosting to Reduce Overfitting

- Trains trees using an ordered data scheme
- Prevents information leakage during training

Native Categorical Feature Handling

- Encodes categorical variables internally

Robust to Missing Data and Nonlinear Effects

- Learns optimal splits for missing values
- Ideal for capturing complex interactions

Model Training & Performance Comparison

```
============================================================
MODEL PERFORMANCE (Out-of-Fold RMSE)
============================================================
CatBoost:       3.3087 kWh  (weight: 0.4)
XGBoost:        3.3333 kWh  (weight: 0.3)
Ridge:          3.5228 kWh  (weight: 0.2)
Random Forest:  3.4912 kWh  (weight: 0.1)
============================================================
ENSEMBLE:       3.3287 kWh
============================================================
```

Out-of-Fold (OOF) Evaluation

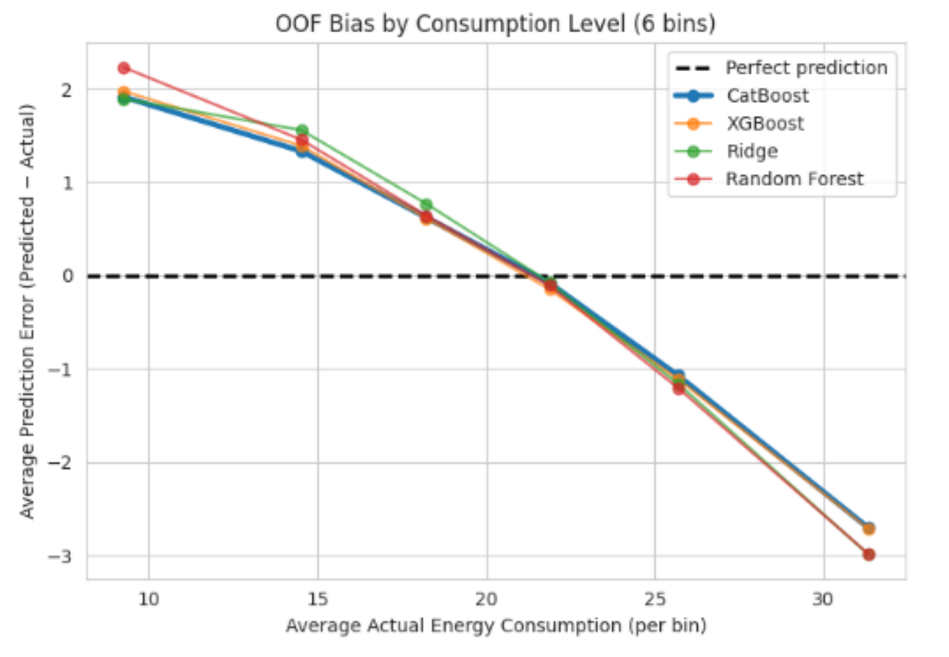
- 5-fold cross-validation used for all models
- Each model evaluated on unseen folds

Performance Comparison

- CatBoost achieved the lowest OOF RMSE
- XGBoost performed competitively but slightly worse
- Ridge and Random Forest underperformed

Takeaway

- CatBoost emerged as the strongest standalone model



$$\mathbb{E}[\hat{y} - y \mid y \in \text{bin}]$$

Limits of Ensembling in This Setting

Highly Correlated Models
- Models learned similar patterns
- Limited diversity reduced ensemble benefit

Marginal Validation Improvements
- Small OOF gains close to noise level
- Did not justify added complexity

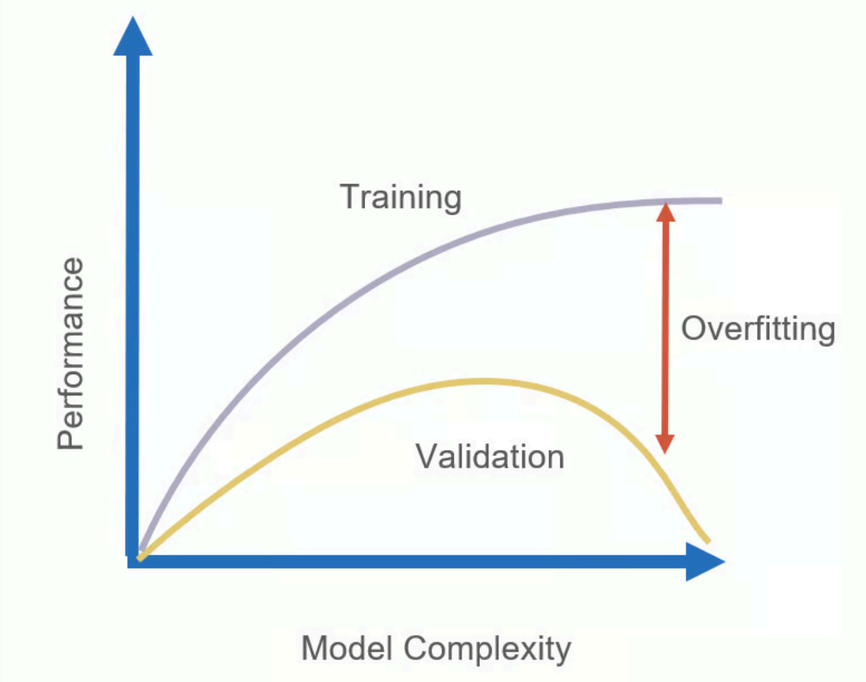
Robustness-Focused Diagnostic
- Conservative CatBoost + linear anchor
- Ranked higher than several of our other models
- Signaled diminishing returns from complexity

Generalization Breakdown
- Ensembles degraded on hidden test data
- Indicated mild overfitting to validation folds

```
CatBoost:        3.3087 kWh
```

```
================================
RIDGE BLENDER: Learned Weights
================================
Intercept: -0.251797

Weights:
  CatBoost:      +0.8789
  XGBoost:       +0.2777
  Ridge:         -0.0615
  Random Forest: -0.0827

Blender OOF RMSE: 3.3044 kWh
================================
```

```
=====================================
2-MODEL BLENDER: CatBoost + XGBoost
=====================================
Intercept: -0.316703

Weights:
  CatBoost: +0.8307
  XGBoost:  +0.1849

2-model Blender OOF RMSE: 3.3061 kWh
=====================================
```



Source: "Validation Learning Curve - Overfitting and Underfitting," CodingNomads

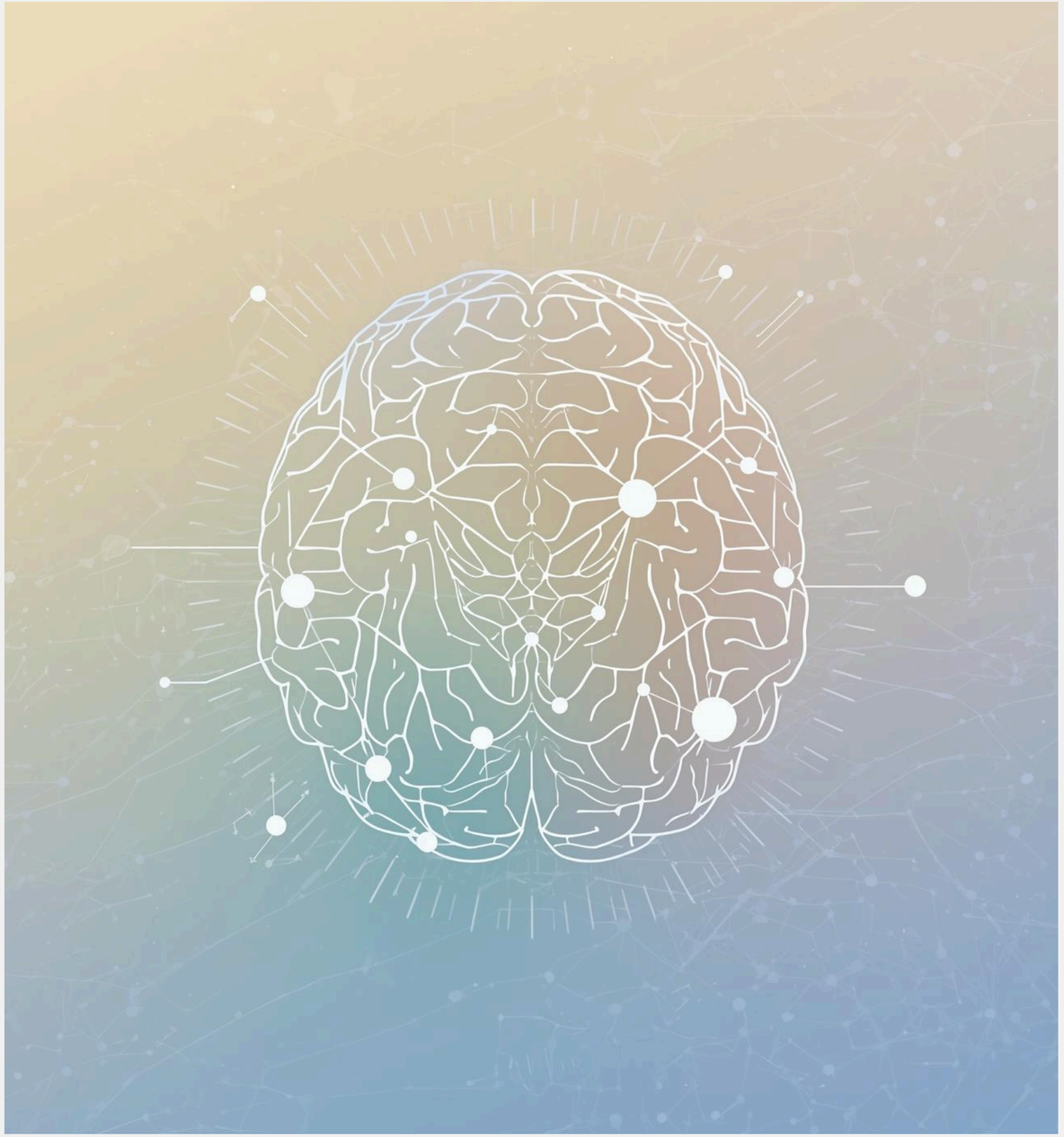
Final Model Choice and Justification

BEST PERFORMANCE ON HIDDEN TEST DATA
CatBoost achieved the lowest RMSE on the hidden test set, outperforming all ensemble variants.

SIMPLER MODEL WITH FEWER DEGREES OF FREEDOM
A single CatBoost model reduces complexity and overfitting compared to multi-model ensembles.

LESS SENSITIVE TO DISTRIBUTION SHIFT
CatBoost generalized better when the test data distribution differed slightly from the training data, while ensembles learned small corrections that did not transfer.

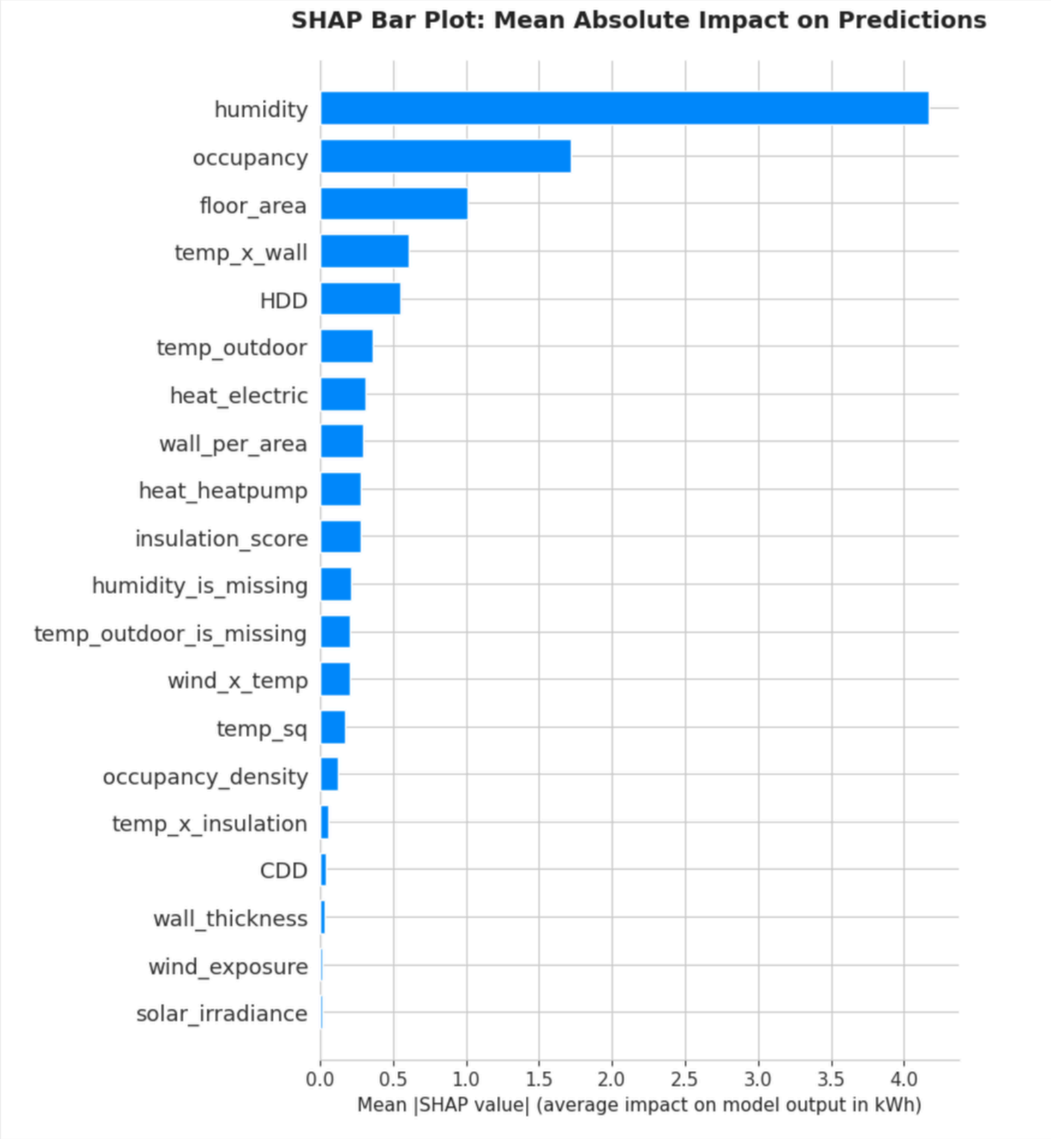
STRONG HANDLING OF MISSING DATA AND CATEGORICAL FEATURES
CatBoost natively incorporates categorical features and missing values, reducing reliance on preprocessing and preserving informative structure in the data.

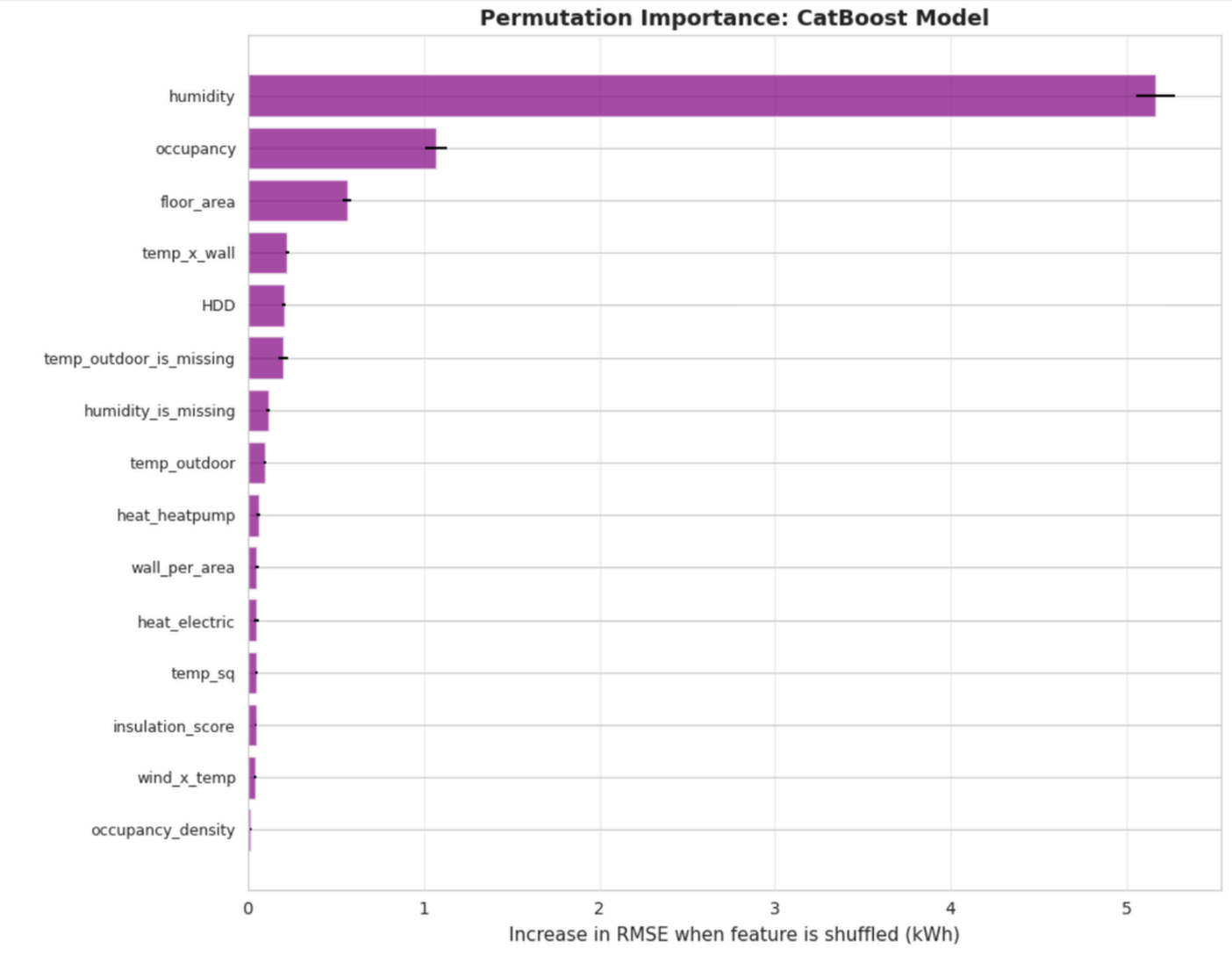


Insights

This section delves into the **key findings** derived from our predictive models. By interpreting the results, we uncover patterns and trends in energy consumption, enabling informed decisions for sustainable energy management and providing valuable insights for future research and applications in this domain.

Feature Importance

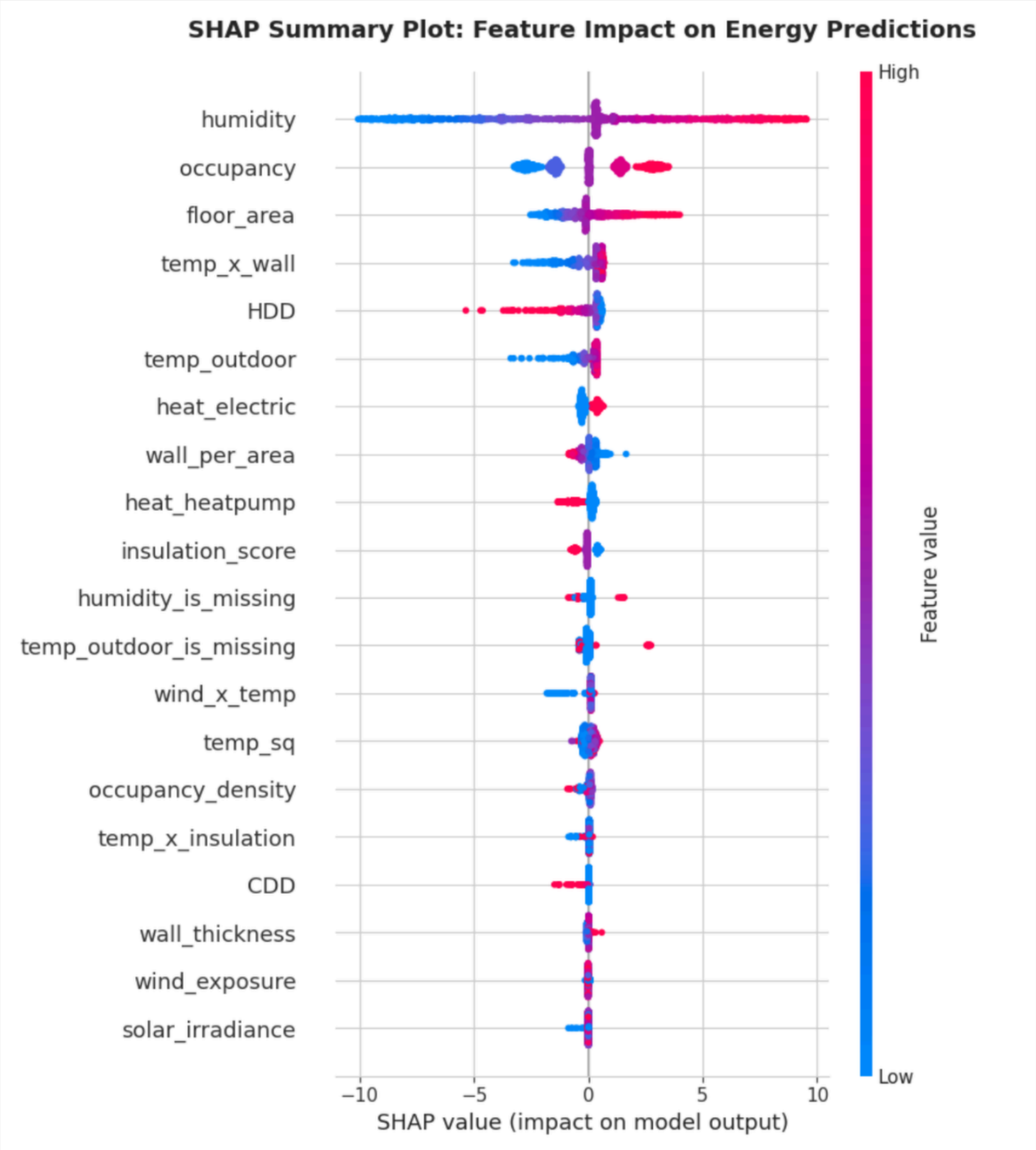




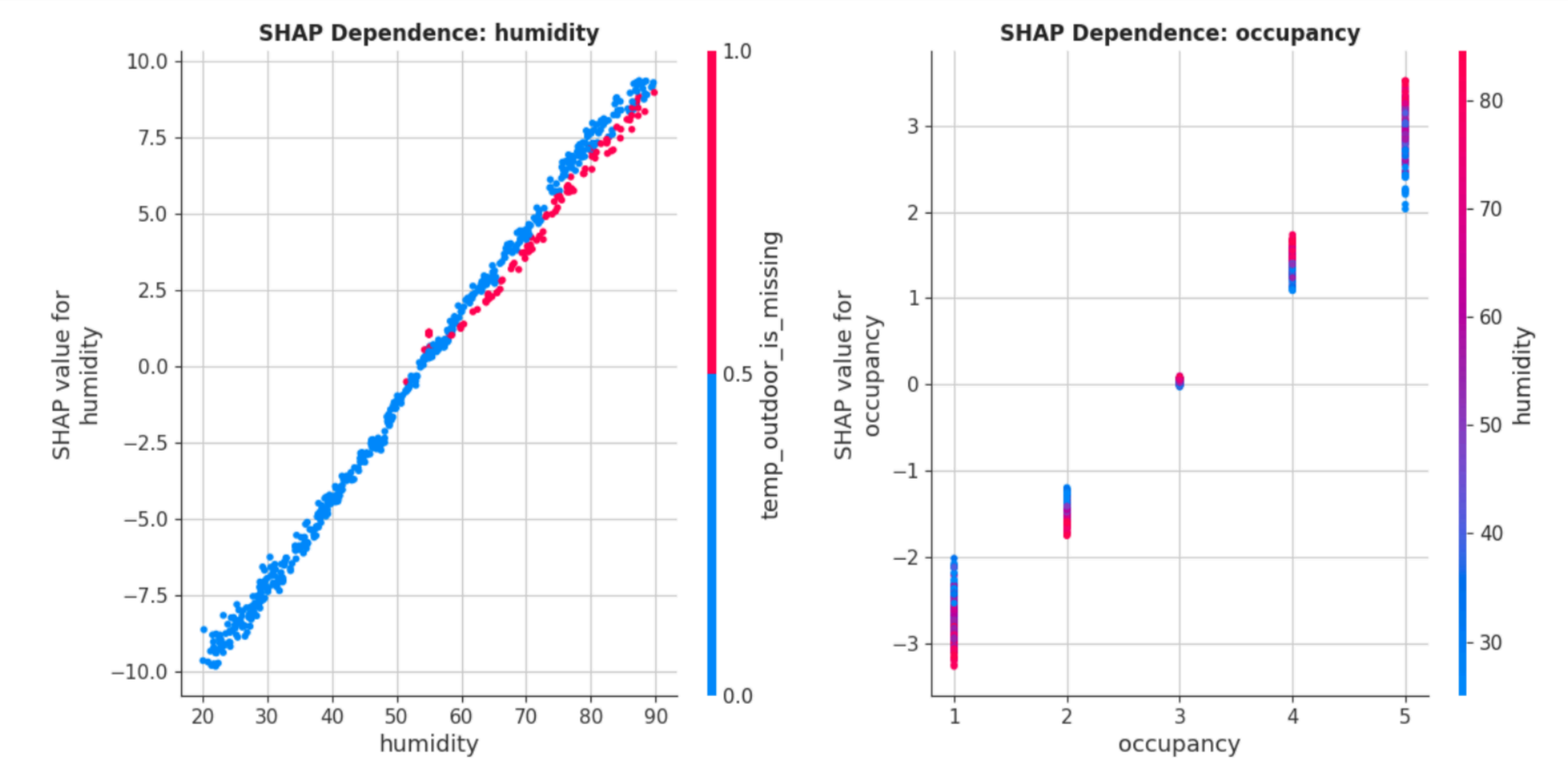
Feature Impact

- calculates each features credit for final score
- x-axis: Impact on output
- Red = high feature value
- Blue = low feature value

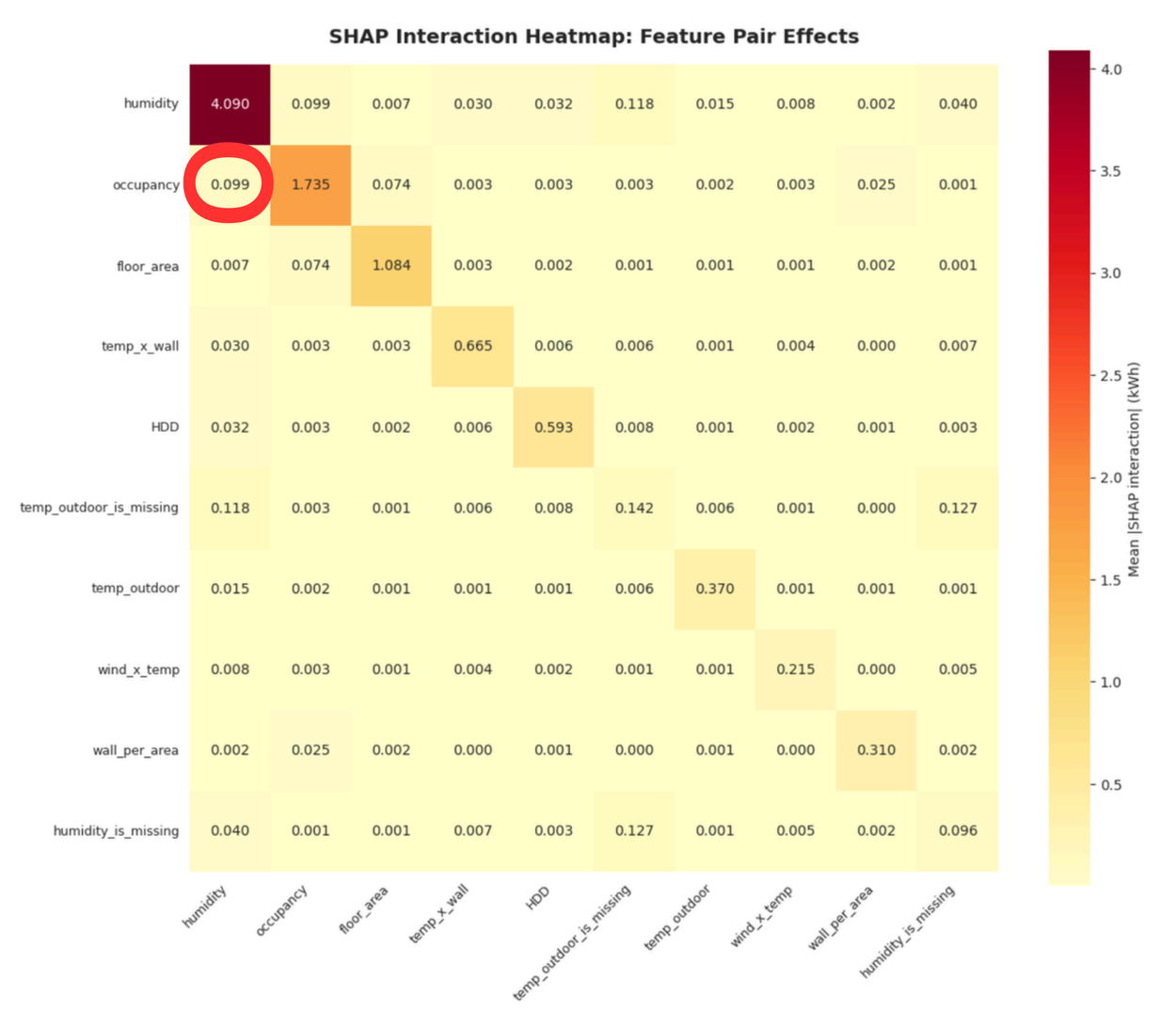
- humidity also most volatile feature
- Vertical clusters dependencies on other features



Feature Dependencies



Feature Interaction



Conclusion and Future Steps

REFLECTING ON OUR DATA SCIENCE PROJECT

Our project highlights the potential of data science approaches for predicting energy consumption. Future research should focus on improving model accuracy and exploring broader datasets for enhanced insights.

Thank You

GROUP 8

Noe
Itzel
Karina
Diren